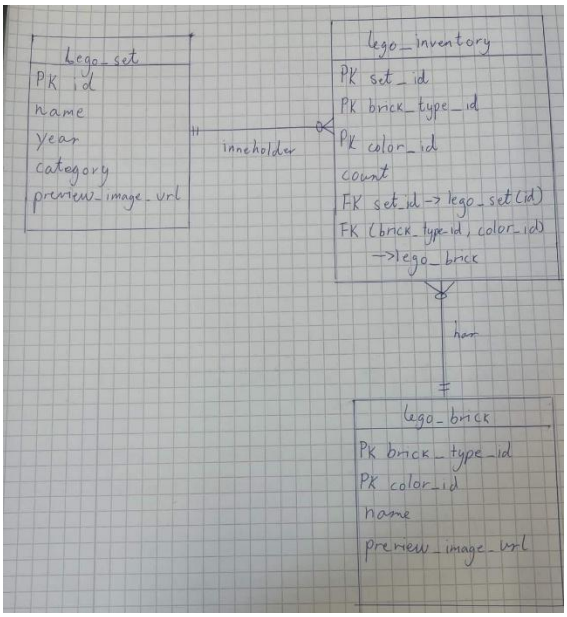
Rapport – DAVE3606

Hanifah Sandow Salifu – hasal0531

Samira Ali Ahmed Elmi- saelm3001

Oppgave 1:



Etter å ha printet ut tabellene så lagde jeg et ER-diagram brutt opp for å vite hvor jeg bør sette primærnøkler og fremmednøkler.

Primary keys:

```
lego-db=# ALTER TABLE lego_set
lego-db-# ADD PRIMARY KEY (id);
ALTER TABLE
```

Hvert lego-set har en unik id, så jeg valgte å sette det som primærnøkkel i lego_set tabellen.

```
lego-db=# ALTER TABLE lego_brick
lego-db-# ADD PRIMARY KEY (brick_type_id, color_id)
lego-db-# ;
ALTER TABLE
```

Jeg lagde en sammensatt primærnøkkel fordi at hver brikketype kan ha ulike farger. Så kombinasjonen av dette gir en unik identifikator.

```
lego-db=# \d lego_inventory
                Table "public.lego_inventory"
    Column      |  Type   | Collation | Nullable | Default
---------------+---------+-----------+----------+---------
 set_id        | text    |           | not null |
 brick_type_id | text    |           | not null |
 color_id      | integer |           | not null |
 count         | integer |           | not null |
Indexes:
    "lego_inventory_pkey" PRIMARY KEY, btree (set_id, brick_type_id, color_id)
```

Her lagde jeg også sammensatt primærnøkkel sånn at en spesifikk brikke går til et spesifikt sett.

Foreign keys:

Inventory → set

ALTER TABLE lego_inventory

ADD FOREIGN KEY (set_id)

REFERENCES lego_set(id);

Inventory → brick

ALTER TABLE lego_inventory

ADD FOREIGN KEY (brick_type_id, color_id)

REFERENCES lego_brick(brick_type_id, color_id);

Fremmednøkler ble lagt til for å sikre referensiell integritet i databasen. Kolonnen set_id i lego_inventory refererer til lego_set(id), mens kombinasjonen (brick_type_id, color_id) refererer til lego_brick. Dette sørger for at alle rader i lego_inventory peker til gyldige sett og brikker.

Primærnøklene er valgt slik at de støtter vanlige spørringer effektivt. Spesielt vil indekseringen gjøre det raskere å hente alle brikker i et bestemt sett (filtrert på set_id), samt å finne hvilke sett som inneholder en bestemt brikketype og farge.

Tabellen lego_inventory inneholdt duplikate rader med identiske verdier for (set_id, brick_type_id, color_id). Disse ble fjernet før primærnøkkelen ble lagt til, slik at hver kombinasjon blir unik og kan fungere som en gyldig sammensatt primærnøkkel.

2:

For å effektivisere spørringer som filtrerer på brikketype og farge, ble det opprettet indekser på kolonnene brick_type_id og color_id i tabellen lego_inventory.

Følgende SQL-setninger ble brukt:

```
CREATE INDEX idx_brick_type
ON lego_inventory(brick_type_id);

CREATE INDEX idx_color
ON lego_inventory(color_id);
```

To spørringer ble testet med \timing on:

Test 1 før indeksering:

```
lego-db=# SELECT COUNT(*)
FROM lego_inventory
WHERE brick_type_id = '3024';
 count
-------
 12364
(1 row)

Time: 167.131 ms
```

Etter indeksering:

- 292.582 ms

Test 2 før indeksering:

```
lego-db=# SELECT COUNT(*)
FROM lego_inventory
WHERE color_id = 1;
 count
--------
 136414
(1 row)

Time: 134.941 ms
```

Etter indeksering:

- 127.924 ms

Resultatene viser at indekser ikke alltid gir bedre ytelse. For spørringen som filtrerer på brick_type_id, økte kjøretiden etter indeksering. Dette skyldes at spørringen returnerer mange rader og dermed er lite selektiv. Selv om indeksen brukes, må databasen fortsatt hente store deler av tabellen, noe som reduserer effekten av indeksen.

```
                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------
---------------------------
 Finalize Aggregate  (cost=11692.13..11692.14 rows=1 width=8) (actual time=117.493..119.521 rows=1.00 loops=1)
   Buffers: shared hit=17 read=7120
   ->  Gather  (cost=11691.92..11692.13 rows=2 width=8) (actual time=117.369..119.515 rows=3.00 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=17 read=7120
         ->  Partial Aggregate  (cost=10691.92..10691.93 rows=1 width=8) (actual time=113.111..113.112 rows=1.00 loops=3)
               Buffers: shared hit=17 read=7120
               ->  Parallel Bitmap Heap Scan on lego_inventory  (cost=1466.72..10555.05 rows=54747 width=0) (actual time=1.546..111
.216 rows=45471.33 loops=3)
                     Recheck Cond: (color_id = 1)
                     Heap Blocks: exact=2336
                     Buffers: shared hit=17 read=7120
                     Worker 0:  Heap Blocks: exact=2334
                     Worker 1:  Heap Blocks: exact=2309
                     ->  Bitmap Index Scan on idx_color  (cost=0.00..1433.87 rows=131392 width=0) (actual time=3.642..3.642 rows=13
6414.00 loops=1)
                           Index Cond: (color_id = 1)
                           Index Searches: 1
                           Buffers: shared read=118
 Planning Time: 0.114 ms
 Execution Time: 119.581 ms
(20 rows)
```

For spørringen som filtrerer på color_id, ble kjøretiden noe lavere etter indeksering. Query plan viser at databasen benytter en Bitmap Index Scan, som gjør det mulig å finne relevante rader mer effektivt før de hentes fra tabellen.

Dette viser at indekser gir best ytelsesforbedring når spørringer er selektive og returnerer få rader, mens effekten er mindre når en stor andel av tabellen matches.

Oppgave 3:

I oppgave 3 analyserte vi ytelsen til endepunktet /sets, som viste seg å være tregt (ca. 3,7 sekunder).

Problemet var at HTML ble bygget ved hjelp av strengkonkatenasjon inne i en løkke. Dette fører til kvadratisk tidskompleksitet, $O(n^2)$, fordi en ny streng blir opprettet for hver iterasjon.

```
127.0.0.1 - - [23/Mar/2026 22:33:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 22:33:10] "GET /favicon.ico HTTP/1.1" 404 -
Time to render all sets: 3.686476425999899
127.0.0.1 - - [23/Mar/2026 22:34:33] "GET /sets HTTP/1.1" 200 -
Time to render all sets: 3.852751628000078
127.0.0.1 - - [23/Mar/2026 22:37:21] "GET /sets HTTP/1.1" 200 -
127.0.0.1 - - [24/Mar/2026 21:14:05] "GET / HTTP/1.1" 200 -
```

For å forbedre ytelsen endret vi implementasjonen til å først samle HTML-radene i en liste, og deretter bruke "".join(...) til å sette dem sammen til én streng. Denne tilnærmingen har lineær tidskompleksitet, O(n).

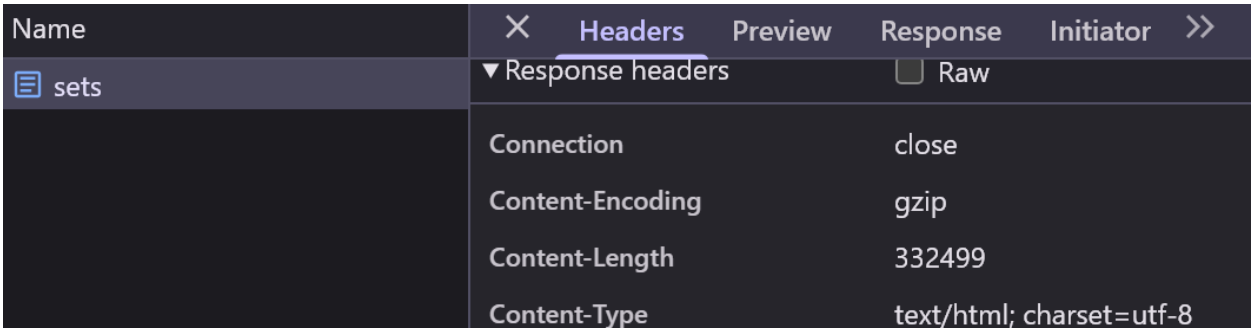
Etter endringen ble responstiden redusert betydelig, fra omtrent 3,7 sekunder til rundt 0,4 sekunder.

```
127.0.0.1 - - [24/Mar/2026 22:10:00] "GET / HTTP/1.1" 200 -
Time to render all sets: 0.4271198349997576
127.0.0.1 - - [24/Mar/2026 22:10:04] "GET /sets HTTP/1.1" 200 -
```

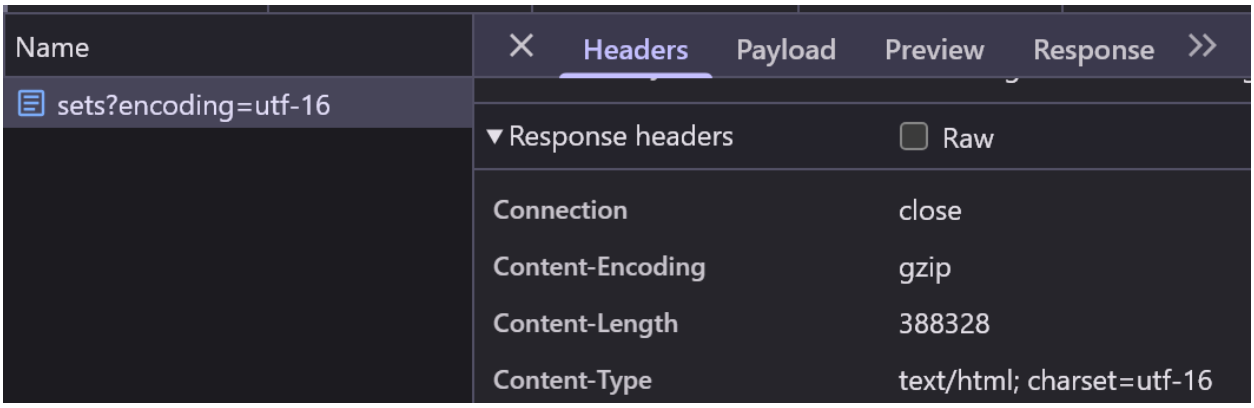
4:

Endpointet /sets ble utvidet til å støtte encoding via en query-parameter (utf-8 som standard og utf-16 som alternativ). Responsen ble kodet med valgt encoding og sendt med riktig HTTP-header. I tillegg ble responsen komprimert med gzip for å redusere datastørrelsen

UTF-8:



UTF-16:



Testing viste at responsen med utf-16 var større (388 328 bytes) enn med utf-8 (332 499 bytes), siden UTF-16 bruker flere bytes per tegn. Gzip reduserer størrelsen på responsen, spesielt for tekstbasert innhold som HTML.

Oppgave 5;

I denne oppgaven skulle jeg implementere to måter å hente ut informasjon om Lego-sett på: først som JSON, og deretter i et eget binært filformat. I tillegg skulle jeg lage et lite konsollprogram som kan lese den binære filen og skrive ut innholdet på en lesbar måte.

Del 1 – JSON-endepunkt

Jeg implementerte endepunktet `/api/set`, som tar imot et set-id som parameter, for eksempel /api/set?id=71799-1. Når dette endepunktet kalles, henter jeg først informasjon om Lego-settet fra tabellen lego_set, og deretter inventaret fra lego_inventory.

For å få med navn og bilde til hver kloss, koblet jeg lego_inventory sammen med lego_brick ved hjelp av brick_type_id og color_id. Resultatet ble strukturert som et JSON-objekt.

JSON-strukturen inneholder:

- informasjon om settet: id, name, year, category, preview_image_url
- en liste inventory
- for hver rad i inventaret: brick_type_id, color_id, count, brick_name, brick_preview_image_url

Jeg brukte parameteriserte SQL-spørringer (%s) for å unngå SQL injection, i stedet for å bygge SQL-strenger manuelt.


```
"id": "71799-1",
"name": "NINJAGO City Markets",
"year": 2023,
"category": "Catalog: Sets: NINJAGO: Dragons Rising Season 1",
"preview_image_url": "https://img.bricklink.com/ItemImage/ST/0/71799-1.t1.png",
"inventory": [
    {
        "brick_type_id": "10170",
        "color_id": 150,
        "count": 3,
        "brick_name": "Medium Nougat Pretzel",
        "brick_preview_image_url": "https://img.bricklink.com/P/150/10170.jpg"
    },
    {
        "brick_type_id": "10187",
        "color_id": 6,
        "count": 6,
        "brick_name": "Green Minifigure, Weapon Bladed Claw, Spread",
        "brick_preview_image_url": "https://img.bricklink.com/P/6/10187.jpg"
    },
    {
        "brick_type_id": "10202",
        "color_id": 11,
        "count": 1,
        "brick_name": "Black Tile 6 x 6 with Bottom Tubes",
        "brick_preview_image_url": "https://img.bricklink.com/P/11/10202.jpg"
    },
    {
```

Del 2 – Binært filformat

Deretter implementerte jeg et nytt endepunkt, /api/set_binary, som returnerer samme informasjon som binær data. Jeg satte content_type til "application/octet-stream" slik at klienten forstår at dette er rå byte-data.

Jeg designet et eget binært filformat. For tekstfelter valgte jeg å lagre:

først lengden på teksten (4 bytes)
deretter selve teksten kodet i UTF-8

For heltall brukte jeg struct.pack for å lagre dem med fast størrelse.

Dataene ble lagret i følgende rekkefølge:

1. set-id
2. set-navn
3. år
4. kategori
5. preview_image_url
6. antall inventory-rader
7. for hver inventory-rad:
    a. brick_type_id
    b. color_id
    c. count
    d. brick_name
    e. brick_preview_image_url

Jeg laget også en hjelpefunksjon pack_string() for å gjøre serialiseringen av tekst enklere og mer konsistent.

```
samira@Lenovo01:~/dave3606-project-2026$ curl "http://127.0.0.1:
5000/api/set_binary?id=71799-1" --output set_binary.bin
ls
  % Total    % Received % Xferd  Average Speed   Time    Time
  Time  Current
                                 Dload  Upload   Total   Spent
  Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:--
  0     0    0     0    0     0      0      0 --:--:-- --:--:--
100  158k  100  158k    0     0   141k      0  0:00:01  0:00:01
--:--:--  141k
README.md                    requirements.txt
bricklink.json.gz            server.py
connect_to_database.sh       set_binary
create_and_run_database.sh   set_binary.bin
import_into_database.py      start_database_after_stopping.sh
migrate_database.py          stop_database.sh
read_set_binary.py           templates
```

Del 3 – Leserprogram

Til slutt laget jeg et Python-program (read_set_binary.py) som leser den binære filen og skriver ut innholdet i terminalen.

Programmet leser dataene i samme rekkefølge som de ble skrevet av serveren. For tekstfelter leser jeg først lengden, og deretter riktig antall bytes, som dekodes til UTF-8. For heltall bruker jeg struct.unpack.

Jeg laget også hjelpefunksjoner:

- read_exact() for å sikre at riktig antall bytes leses
- unpack_string() for å lese tekstfelt

Dette gjør programmet mer robust og enklere å forstå.

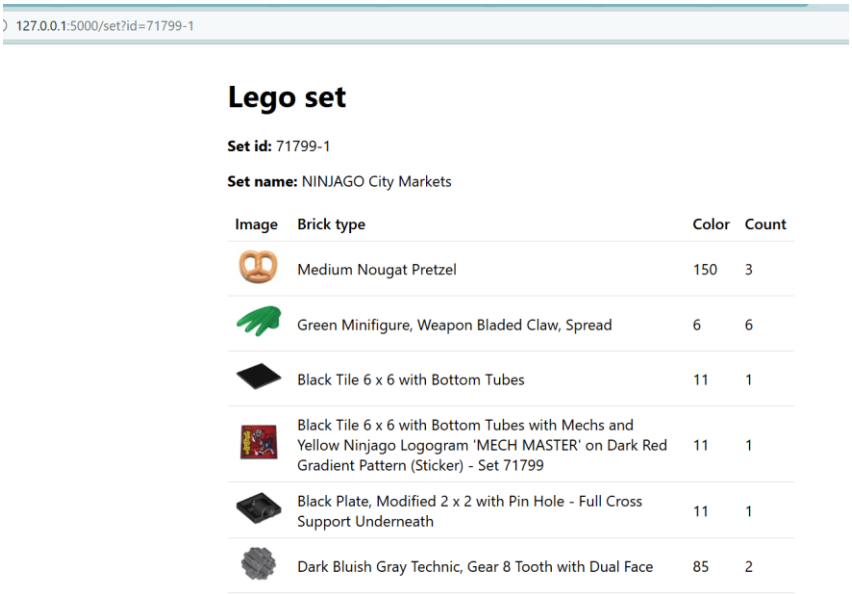
```
Entry 1101:
  Brick type ID: 63864
  Color ID: 86
  Count: 5
  Brick name: Light Bluish Gray Tile 1 x 3
  Brick preview image URL: https://img.bricklink.com/P/86/63864.
jpg

Entry 1102:
  Brick type ID: 63864
  Color ID: 110
  Count: 1
  Brick name: Bright Light Orange Tile 1 x 3
  Brick preview image URL: https://img.bricklink.com/P/110/63864
.jpg

Entry 1103:
  Brick type ID: 63864
  Color ID: 120
  Count: 11
  Brick name: Dark Brown Tile 1 x 3
  Brick preview image URL: https://img.bricklink.com/P/120/63864
.jpg
```

Oppgave 6;

I oppgave 6 implementerte jeg frontend for visning av LEGO-sett ved hjelp av JavaScript. Siden henter data fra /api/set-endepunktet og oppdaterer DOM-en dynamisk. Set-id, navn og inventar vises i en tabell med bilde, navn, farge og antall.

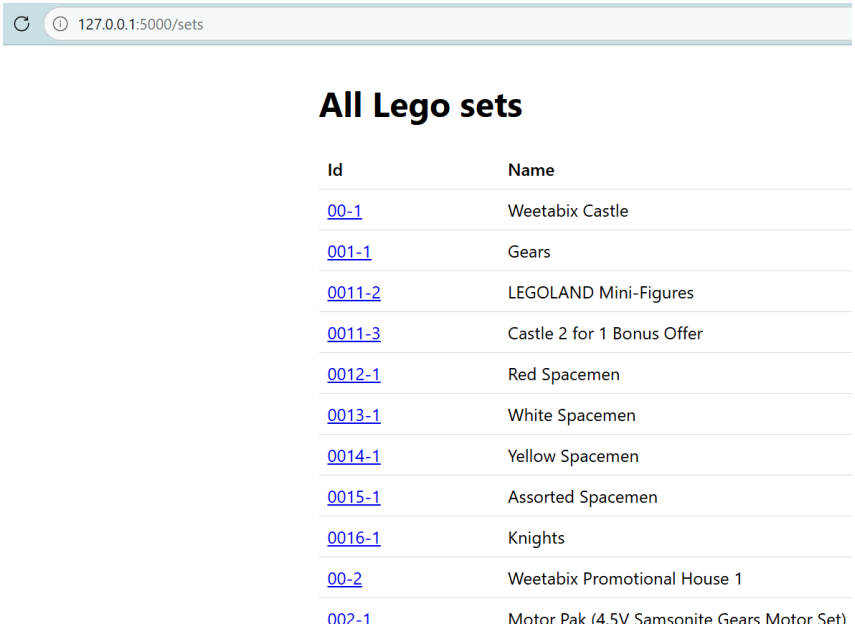


Jeg implementerte også en server-side cache for /api/set ved hjelp av OrderedDict. Cachen lagrer de 100 sist brukte settene. Ved cache-treff returneres data direkte uten databaseoppslag, mens ved cache-miss hentes data fra databasen og lagres i cachen. Når cachen overstiger 100 elementer, fjernes det eldste elementet.

```
DB_CONFIG = {
    "host": "localhost",
    "port": 9876,
    "dbname": "lego-db",
    "user": "lego",
    "password": "bricks",
}

SET_CACHE = OrderedDict()
CACHE_LIMIT = 100
```

I tillegg la jeg til Cache-Control-header på /sets-responsen slik at nettleseren kan mellomlagre siden i 60 sekunder. Dette reduserer antall forespørsler og forbedrer ytelsen.



Oppgave 7:

I denne oppgaven refaktorerte jeg endpointene for å gjøre koden mer testbar. Jeg flyttet mesteparten av logikken ut av endpoint-funksjonene og inn i egne funksjoner, slik at endpointene kun håndterer forespørsler og returnerer en Response. Dette gjør koden mer oversiktlig og lettere å jobbe videre med.

Jeg laget også en Database-klasse som fungerer som et lag over psycopg. Denne håndterer oppkobling til databasen, kjøring av SQL-spørringer og lukking av tilkoblingen. Ved å sende inn et Database-objekt til funksjonene (dependency injection), unngår jeg å ha databasen hardkodet direkte i logikken.

Dette gjorde det mulig å teste funksjonene uten å bruke en ekte database. I testene brukte jeg derfor en mock-database som returnerer forhåndsdefinerte data og sjekker at riktig SQL-spørring blir brukt. Deretter testet jeg at funksjonene gir riktig HTML- og JSON-output.

```
(venv) hanifah@Shadow:~/dave3606-project-2026$ python3 -m pytest
============================================= test session starts =============================================
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/hanifah/dave3606-project-2026
collected 2 items

test_server.py ..                                                                                        [100%]

============================================== 2 passed in 0.16s ==============================================
```

Testene ble kjørt med pytest, og begge testene besto (2 passed). Dette viser at funksjonene fungerer korrekt med mock-database og genererer riktig output.